

6CM504 Concurrency and communication

Modelling and verification

Prof. Alistair A. McEwan

School of Computing
University of Derby

Table of Contents

- 1 Introduction
- 2 And so, to refinement and specification
- 3 Specifications
- 4 Refinement
- 5 My first CSP specification and verification
- 6 Summary

This week

Last week, we looked at process traces—one of the semantic models that gives meaning to CSP processes. We saw how process traces are sets of (possibly empty) sequences of events that describe the observable behaviours of a process. This week we will look further into process traces to understand relationships between apparently similar processes. These relationships will allow us to construct *specifications*—descriptions of the behaviours that we want from our systems.

Table of Contents

- 1 Introduction
- 2 And so, to refinement and specification
- 3 Specifications
- 4 Refinement
- 5 My first CSP specification and verification
- 6 Summary

What is refinement?

- 1 Refinement: an advance towards something more refined; an improvement, modification, or clarification of something by the making of small changes. Also: a state arrived at, or thing obtained, in this way.
- 2 Refinement: Generally: the process of refining or purifying something. Also: the result of refining, or an instance of this; the state of being refined.

(Oxford English Dictionary)

What is specification?

- ① Specification: The action of investing with some specific or determinate quality; conversion to something specific.
- ② Specification: The formation of a new species of property out of material belonging to another by converting it into a different form.

(Oxford English Dictionary)

First, a little more on sequences...

- 1 There are a number of laws and operations that can be applied to sequences
- 2 We will use some of these to manipulate process traces (and sets of process traces) in order to reason about them
- 3 Over the next few slides we will revise a few key properties of sequences that will allow us to talk about refinement and specifications

Restriction

- 1 The operator $\upharpoonright$ is known as the restriction operator
- 2 The expression $t \upharpoonright A$ denotes the trace t when restricted to symbols in the set A
- 3 For example: $\langle a, b, b, d, a, c \rangle \upharpoonright \{b, d, c\} = \langle b, b, d, c \rangle$

Restriction: some laws

1 Restriction distributes

$$\langle \rangle \upharpoonright A = \langle \rangle$$

$$(s \frown t) \upharpoonright A = (s \upharpoonright A) \frown (t \upharpoonright A)$$

2 For example if $s = \langle a, d, b \rangle$ and $t = \langle b, b, d, a \rangle$ then

$$(s \frown t) \upharpoonright \{d, b\} = \langle d, b, b, b, d \rangle$$

Restriction and set operations

- 1 A trace restricted to the empty set of symbols leaves nothing

$$s \upharpoonright \{\} = \langle \rangle$$

- 2 A successive restriction by two sets is the same as a single restriction by the union of those two sets

$$(s \upharpoonright A) \upharpoonright B = s \upharpoonright (A \cup B)$$

Sequence length

- 1 The expression

$\#A$

denotes the length of the sequence A

- 2 For example

$$\#\langle \rangle = 0$$

$$\#\langle a, a, b, a, c \rangle = 5$$

Sequence length and restriction

- 1 The expression

$$\#(A \upharpoonright B)$$

gives the length of the sequence A when restricted to elements in the set B

- 2 For convenience we abbreviate this to the symbol

$$A \downarrow B$$

Table of Contents

- 1 Introduction
- 2 And so, to refinement and specification
- 3 Specifications**
- 4 Refinement
- 5 My first CSP specification and verification
- 6 Summary

Example specifications

- 1 A specification of a product is a description of the way it is intended to behave, covering some observable aspect of the behaviour of the product
- 2 We are going to study how to write a specification for a system, and how to prove that a model meets the requirements of a specification

Example specifications

- 1 The owner of our vending machine does not wish to make a loss by installing it. He therefore specifies that the number of chocolates it dispenses must never exceed the number of coins inserted

$$NoLoss = \#(tr \upharpoonright \{choc\}) \leq \#(tr \upharpoonright \{coin\})$$

Example specifications

- 1 The customers of the vending machine wants to ensure that it will not absorb further coins until it has dispensed the chocolate already paid for

$$Fair = (tr \downarrow \{coin\}) \leq (tr \downarrow \{choc\}) + 1$$

Example specifications

- 1 The manufacturer of the simple vending machine realises he must meet the requirements of both the owner and the customer

$$VMSpec = NoLoss \wedge Fair$$

$$= (0 \leq ((tr \downarrow \{coin\}) - (tr \downarrow \{choc\}))) \leq 1)$$

Example specifications

- 1 Our complicated vending machine had a flaw in it—it stopped working if three 50 pence pieces were inserted in a row
- 2 Instead of just putting a sign on it, we could have specified that this behaviour should never have been included in the first place

$$FixVM = \neg (\langle in50p \rangle^3 \text{ in } tr)$$

Example specifications

- 1 We can now exactly give the specification of the observable behaviours we would like of our vending machine

$$MendedVM = VMSpec \wedge FixVM$$

- 2 Or more specifically, we can give the specification we wanted for our original machine

$$MendedVM = (tr \in traces(ComplicatedVM) \wedge FixVM)$$

Satisfaction!

- 1 If P is a product which meets a specification S , we say that P *satisfies* S , abbreviated to

$$P \text{ sat } S$$

- 2 This means that every possible observation of P is described by S
 - In other words, S is true whenever its variables take values observed from the product P
- 3 More formally

$$\forall tr \bullet tr \in \text{traces}(P) \Rightarrow S$$

Specification

- ① Real-world systems are designed to satisfy particular requirements
 - One of the uses of semantics is to allow them to be judged against particular specifications
- ② We have looked at process traces
 - This gives a meaning to CSP processes in terms of describing all the possible observations that may be made of a process
 - We refer to this as “the traces model”
- ③ A specification is simply a description of required behaviour
 - The specifications we have looked at so far were given in terms of a requirement on the traces a process could perform.

Table of Contents

- 1 Introduction
- 2 And so, to refinement and specification
- 3 Specifications
- 4 Refinement**
- 5 My first CSP specification and verification
- 6 Summary

Refinement

- 1 Another way of describing a set of acceptable traces is in terms of another CSP process
 - A CSP process description corresponds to a set of traces
 - If this set of traces is defined very precisely, then the process can itself act as a specification.

- 2 For instance, the process

$\text{RUN}(\{a, b\})$

has as its traces all sequences containing only a and b events

- 3 As a specification, it captures the requirement that only a and b are allowed, and in any combination

Refinement

- 1 A process $P1$ meets a specification $P0$ if any possible trace of $P1$ is also a valid trace of $P0$
- 2 We say " $P1$ is a refinement of $P0$ "
- 3 For instance given

$$P1 = a \rightarrow b \rightarrow P1$$
$$P0 = \text{RUN}(\{a, b\})$$

then $P1$ meets the specification $P0$ as it performs no events other than a and b

Refinement: notation

- 1 We write this as $P_0 \sqsubseteq T= P_1$
- 2 This is pronounced "*P1 trace-refines P0*"
- 3 Informally this means that every possible observable behaviour of P_1 is acceptable to P_0
- 4 Formally: $P_0 \sqsubseteq T= P_1 = \text{traces}(P_1) \subseteq \text{traces}(P_0)$

Refinement: why?

- ① Why do we care about refinement?
- ② Refinement allows us to show that a given process meets a certain specification
 - A customer who asks for something that respects P_0 will be happy if P_1 is delivered
- ③ In order to show that a given process meets a certain specification we simply have to prove the refinement relation holds
 - This is why we use FDR!
- ④ Refinement is the key notion that we will use for system verification
 - We express a property that we are interested in as a specification process
 - For instance a safety property
 - And test the refinement relation holds with the implementation process

Some refinement laws

- 1 Under refinement, a process is equivalent to itself

$$P \sqsubseteq P \quad ([=-\text{reflex}])$$

- 2 Refinement is transitive: if $P1$ is a refinement of $P0$, and $P2$ is a refinement of $P1$, then $P2$ is a refinement of $P0$

$$(P0 \sqsubseteq P1) \ \&\& \ (P1 \sqsubseteq P2) \Rightarrow (P0 \sqsubseteq P2) \\ ([=-\text{trans}])$$

- 3 This allows for the stepwise development of systems

Some refinement laws

- 1 The process RUN is traces-refined by all other processes (care needs to be taken with alphabets here)

$$\text{RUN}(X) \quad [T = P(X) \quad ([T = \text{bottom}])$$

- 2 Every process is traces-refined by STOP

$$P \quad [T = \text{STOP} \quad ([T = \text{top}])$$

- 3 Question: can you explain why this is the case?

Refinement and non-determinism

- 1 Consider the following process

$$P = P0 \mid \sim \mid P1$$

- 2 This process can non-deterministically behave either as $P0$ or as $P1$
 - The choice is made internally before any observation is made, and the user has no influence over the choice
- 3 If a customer had asked for P , they would therefore be happy to be given either $P0$ or $P1$
 - In fact they would never even know the difference
- 4 Therefore we can make the following two assertions

$$P \sqsubseteq T = P0$$

$$P \sqsubseteq T = P1$$

- 5 Refinement has resolved the nondeterminism!

Table of Contents

- 1 Introduction
- 2 And so, to refinement and specification
- 3 Specifications
- 4 Refinement
- 5 My first CSP specification and verification**
- 6 Summary

A case study

- 1 Assume the alphabet $\text{Meats} = \{\text{raw}, \text{cooked}, \text{wash}\}$ refers to the actions of handling raw meat, handling cooked meat, and washing hands respectively
- 2 Question: can you construct a specification that enforces the hygiene requirement that hands should always be washed between handling raw and cooked meat?

Be careful what you wish for!

- ① Question: what can we say about the following specifications?

`Spec1 = raw -> washed -> cooked -> Spec1`

<code>Spec1 =</code>	<code>Wash = wash -> Spec2</code>
<code>raw -> Wash</code>	
<code>[]</code>	
<code>cooked -> Wash</code>	

Be careful what you wish for!

- ❶ Question: what can we say about the following specifications?

`Spec1 = raw -> washed -> cooked -> Spec1`

<code>Spec1 =</code>	<code>Wash = wash -> Spec2</code>
<code>raw -> Wash</code>	
<code>[]</code>	
<code>cooked -> Wash</code>	

- ❷ Spec1 looks to work—but what about the sequence $\langle \textit{cooked}, \textit{raw} \rangle$
- ❸ Spec2 looks to enforce between both
- ❹ Which is the specification we really want?

Know what you wish to claim!

- ❶ Question: which of the following statements (assertions) are true?
- ❶ $\text{RUN}(\text{Meats}) \text{ } [T = \text{Spec1}]$
 - ❷ $\text{Spec1} \text{ } [T = \text{RUN}(\Sigma)]$
 - ❸ $\text{STOP} \text{ } [T = \text{Spec2}]$
 - ❹ $\text{Spec2} \text{ } [T = \text{STOP}]$
 - ❺ $\text{Spec1} \text{ } [T = \text{Spec2}]$
 - ❻ $\text{Spec2} \text{ } [T = \text{Spec1}]$

Table of Contents

- 1 Introduction
- 2 And so, to refinement and specification
- 3 Specifications
- 4 Refinement
- 5 My first CSP specification and verification
- 6 Summary**

Summary

- 1 In this lecture, we were introduced to the theory of refinement, allowing us to express relationships between different processes
- 2 We expanded this to specifications—descriptions of behaviours that we may or may not want of our systems
- 3 Refinement was shown to be a tool for checking conformance to specifications
- 4 In the coming weeks, we shall use FDR to check refinement relations and investigate specifications
- 5 In the next lecture we will look at another key aspect of CSP—process termination
- 6 This will later allow us to enter the world of concurrency

Summary

- ➊ Advance reading: Schneider, pages 29–42
- ➋ Advance reading: Roscoe, pages 139–147
- ➌ Advance reading: Woodcock and Davies, chapter 7